

Analytics Programming Week-2

2024-08-30

```
#Name: Rupa Shrestha  
#Date: 30th August 2024  
#"CSDA 5110 Homework 2"
```

#Question 2: Which (if any) of the following variable's names is/are invalid?

```
thisone <- 1  
THISONE <- 2  
#1This <- 3    =INVALID  
this.one <- 4  
This.1 <- 5  
ThIS.....ON...E <- 6  
#This!On!e <- 7 =INVALID  
lkjasdfkjsdf <- 8
```

#Invalid Variables are: 3(1This <- 3) and 7(This!On!e <- 7)

#Question 3: 2021 was a good year for the price of Gold – your investment portfolio made \$100,800 from investing in gold. Create a variable called gold.in.2021 and assign the value of your investment portfolio to it.

```
gold.in.2021 <- 100800  
gold.in.2021  
  
## [1] 100800
```

#Question 4: During the last bank audit, we discovered that there was an error caused by a system calculation error and \$5000 per year wasn't added to your gold investment portfolio for the years 2018, 2019, and 2020. Create a new variable called calcError and assign it the amount miscalculated by the system. Create another variable called corrected.total and add the variables gold.in.2021 and calcError to this new variable. What is the new total amount?

```
calcError = 5000  
corrected.total = gold.in.2021 + calcError  
  
print(paste("The new total Value is: ", corrected.total))  
  
## [1] "The new total Value is: 105800"
```

#Question 5: . Create a variable called numberVector1 and numberVector2. Using the c(), assign a vector of numbers, 1 through 15 to each vector. Use a different method of creating the vector for each variable. (Hint: this was covered in your reading of chapter 4) Use the appropriate function to check the length of each of the vector variables you created above

```

numberVector1 <- c(1:15)
numberVector2 <- seq(1,15)

print(length(numberVector1))

## [1] 15

print(length(numberVector2))

## [1] 15

```

#Question 6: Using the two vector variables created above, add them together and assign the total to a variable called totalVectors.

```

totalVectors = numberVector1 + numberVector2
print(totalVectors)

## [1] 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30

```

#Question 7: Use the appropriate function to sum the totalVectors variable to display the sum of the totalVectors variable.

```

print(sum(totalVectors))

## [1] 240

```

#Question 8: Joanne thinks that she completes more tasks when she's had a cup of coffee than when she doesn't. To test this, she recorded how many tasks she completed over 7 days without drinking any coffee, and then did the same over 7 days while drinking coffee. Her results are stored in a comma delimited CSV file called Joannes Tasks #a. Import the comma delimited CSV file into R and assign the data to a variable called joanneTasks #b. Verify the variable holding the data is a data.frame variable #c. Using the appropriate function, write the code to verify that there are 7 rows and 4 columns in the data frame.

```

#A solution
joanneTasks = read.csv("D:/Webster University/Assignments/Analytics
Programming/Week-2/Joannes Tasks.csv")

#B solution
print(paste("The data is of type:", class(joanneTasks)))

## [1] "The data is of type: data.frame"

#C Solution
print(paste("Number of rows", nrow(joanneTasks), "Number of columns",
ncol(joanneTasks)))

## [1] "Number of rows 8 Number of columns 4"

```

#Question 9: Use the appropriate function that will display summary descriptive statistics about each column within the data frame. #a. In your code, create comments about information you deduced from the summary descriptive statistics.

```
summary(joanneTasks)
```

```
## Day.of.the.Week    Number.of.Tasks.With.Coffee
Number.of.Tasks.Without.Coffee
## Length:8           Min.    : 3.000           Min.    : 5.000
## Class :character   1st Qu.: 5.000           1st Qu.: 5.000
## Mode  :character   Median : 6.000           Median : 8.000
##                               Mean  : 6.429           Mean  : 7.714
##                               3rd Qu.: 8.000           3rd Qu.: 8.000
##                               Max.   :10.000          Max.   :15.000
##                               NA's    :1              NA's    :1
## Total.Tasks
## Min.    : 8.00
## 1st Qu.:10.00
## Median :14.00
## Mean    :14.14
## 3rd Qu.:17.00
## Max.    :23.00
## NA's    :1
```

*# Summary for days of the week: Days of the week is character data type.
Length of days of the week is 8.*

*#Summary for the Number.of.Tasks.With.Coffee: Minimun task Joanne completed
with coffee is 3 and maximum value is 10. The average value is 6.429 and
median value is 6. The first quatile value with coffee is 5 and the third
quartile value with coffee is 8. There is one NA value present.*

*#Summary for the Number.of.Tasks.Without.Coffee: Minimun task Joanne
completed without coffee is 5 and maximum value is 15. The average value is
7.714 and median value is 8. The first quatile value without coffee is 5 and
the third quartile value without coffee is 8. There is one NA value present.*

*#Total.Tasks: Minimun total task completed by Joanne is 8 and maximum value
is 23. The average value is 14.14 and median value is 14. The first quatile
value of total task is 10 and the third quartile value of total task is 13.
There is one NA value present.*

#.10 Using the appropriate syntax write the code that allows you to access the column that contains the Number of Tasks Completed Without Coffee data. #a. Use the appropriate function to display the mean value of that column. #b. Use the appropriate function to display the total of the values in the column.

```
task.completed.without.coffee <- (joanneTasks$Number.of.Tasks.Without.Coffee)
print(task.completed.without.coffee)
```

```
## [1] 15  5  8  8  5  5 NA
```

#10 a Solution

```
without_coffee_task_mean <- mean(task.completed.without.coffee, na.rm = TRUE)
print(paste("mean",without_coffee_task_mean))
```

```
## [1] "mean 7.71428571428571"
```

#10 b Solution

```
without_coffee_task_sum <- sum(task.completed.without.coffee, na.rm = TRUE)  
print(paste("sum", without_coffee_task_sum))
```

```
## [1] "sum 54"
```

#Question 11: # Using the appropriate syntax write the code that allows you to access the column that contains the Number of Tasks Completed With Coffee data. #a. Use the appropriate function to display the mean value of that column. #b. Use the appropriate function to display the total of the values in the column.

```
task.completed.with.coffee <- (joanneTasks$Number.of.Tasks.With.Coffee)  
print(task.completed.with.coffee)
```

```
## [1] 8 5 8 6 10 3 5 NA
```

#11 a Solution

```
with_coffee_task_mean = mean(task.completed.with.coffee, na.rm = TRUE)  
print(paste("Mean", with_coffee_task_mean))
```

```
## [1] "Mean 6.42857142857143"
```

#11 b Solution

```
with_coffee_task_sum = sum(task.completed.with.coffee, na.rm = TRUE)  
print(paste("Sum", with_coffee_task_sum))
```

```
## [1] "Sum 45"
```

#12 Using the appropriate syntax write the code that allows you to access the column that contains the Total Daily Tasks data. #a. Use the appropriate function to display the mean value of that column. #b. Use the appropriate function to display the total of the values in the column.

```
total.daily.tasks <- (joanneTasks$Total.Tasks)  
print(total.daily.tasks)
```

```
## [1] 23 10 16 14 18 8 10 NA
```

#12 a Solution

```
total.daily.tasks_mean = mean(total.daily.tasks, na.rm = TRUE)  
print(paste("Mean", total.daily.tasks_mean))
```

```
## [1] "Mean 14.1428571428571"
```

#12 b Solution

```
total.daily.tasks_sum = sum(total.daily.tasks, na.rm = TRUE)  
print(paste("Sum", total.daily.tasks_sum))
```

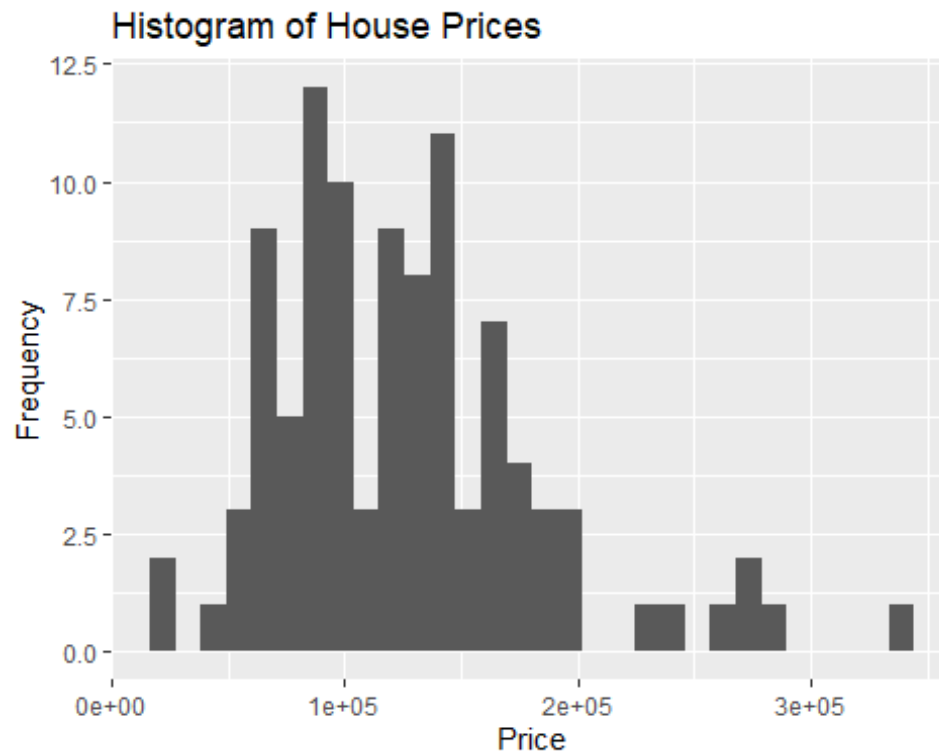
```
## [1] "Sum 99"
```

#13 Write a function that will display the question below and return the correct answer.
#a. Was Joanne able to complete more tasks when she drank coffee or days when she did not drink coffee?

```
compare_task_function <- function(task) {  
  if (without_coffee_task_sum > with_coffee_task_sum) {  
    return("Joanne was able to complete more tasks without coffee than with  
coffee.")  
  } else if (without_coffee_task_sum < with_coffee_task_sum) {  
    return("Joanne was able to complete more tasks with coffee than without  
coffee.")  
  } else {  
    return("Joanne completed the same number of tasks with or without  
coffee.")  
  }  
}  
  
print(compare_task_function(joanneTasks))  
  
## [1] "Joanne was able to complete more tasks without coffee than with  
coffee."
```

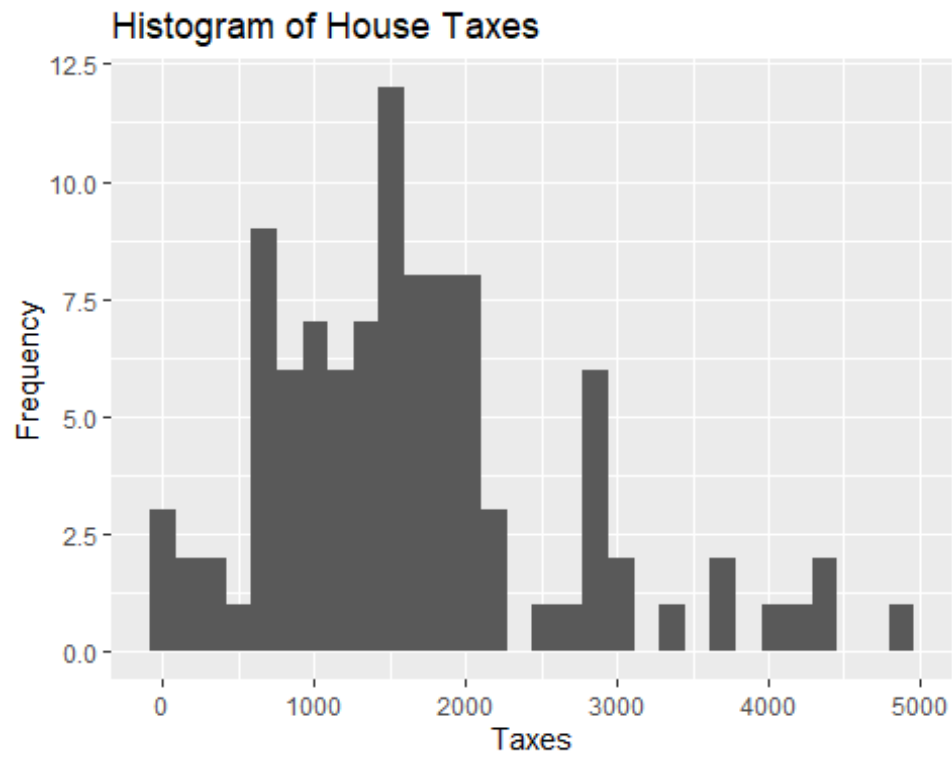
#14. Select two quantitative variables (from Taxes, Price, Size, and Lot) #a. Draw histogram for the two selected quantitative variables. #b. Draw the density plot for the two selected quantitative variables.

```
library(ggplot2)  
library(readxl)  
  
housing_data <- read_excel("D:/Webster University/Assignments/Analytics  
Programming/Week-2/house_selling_prices_FL.xlsx")  
  
#Solution for 14 a  
  
ggplot(housing_data, aes(x=price)) +geom_histogram()+  
  ggtitle("Histogram of House Prices") +  
  xlab("Price") +  
  ylab("Frequency");  
  
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



```
ggplot(housing_data, aes(x=Prices)) +geom_histogram()+  
  ggtitle("Histogram of House Prices") +  
  xlab("Prices")+  
  ylab("Frequency");
```

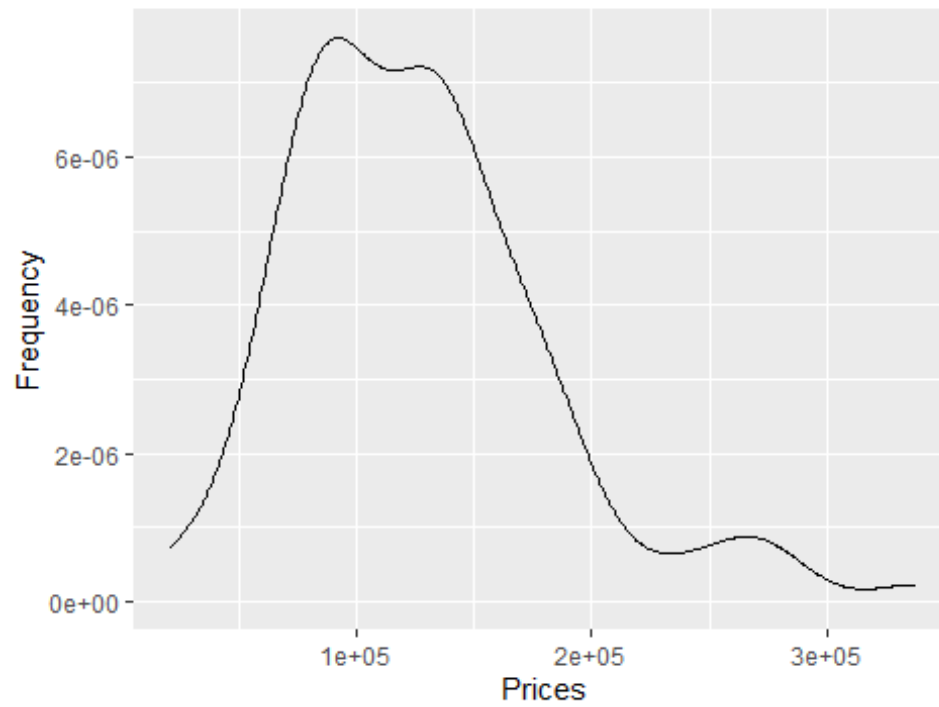
```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



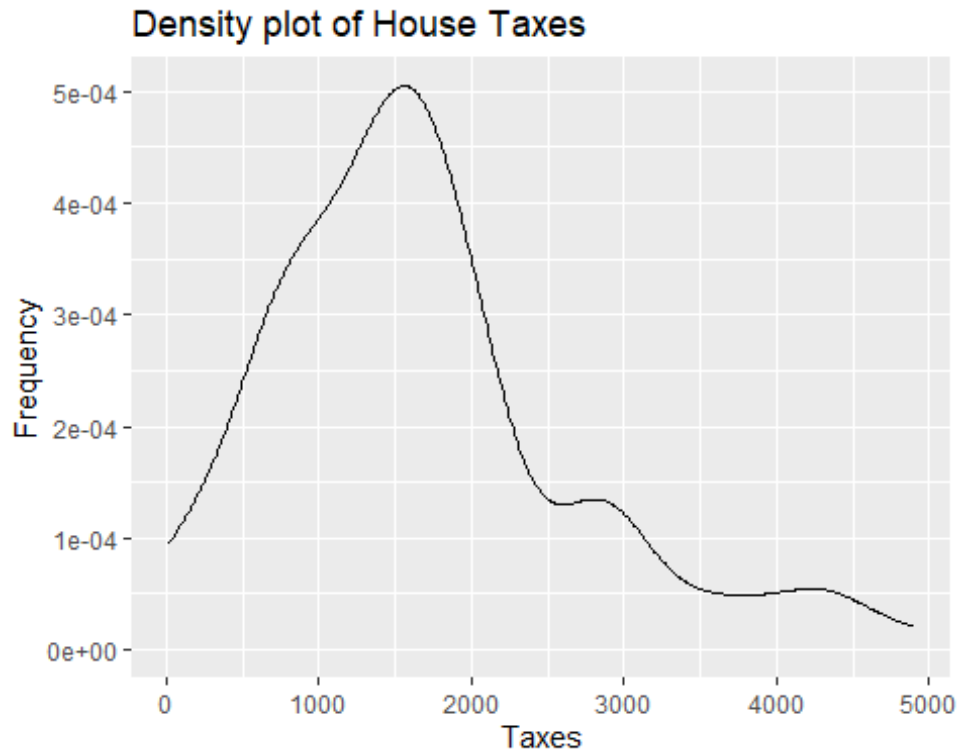
#Solution for 14 b

```
ggplot(housing_data, aes(x=price)) +geom_density()+  
  ggtitle("Density plot of House price") +  
  xlab("Prices") +  
  ylab("Frequency");
```

Density plot of House price



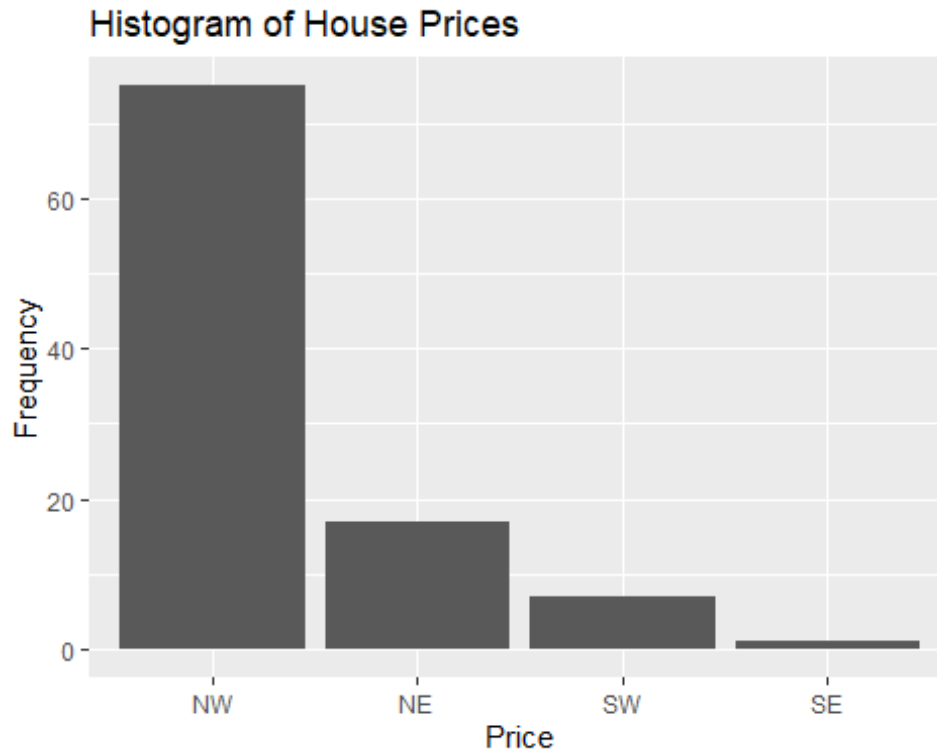
```
ggplot(housing_data, aes(x=Prices)) +geom_density()+  
  ggtitle("Density plot of House Prices") +  
  xlab("Prices") +  
  ylab("Frequency");
```

#15. Select the

categorical variable Quadrant. #a. Draw Pareto chart.

```
library(forcats)
ggplot(housing_data, aes(x=fct_infreq(Quadrant))) + geom_bar() +
  ggtitle("Histogram of House Prices") +
  xlab("Price") +
  ylab("Frequency");
```



#16. For both the quantitative variables used in question 14. #a. Determine the mean and median. #b. Compute the standard deviation of both variables. #c. Compute five number summary for both the variables. #d. Draw boxplots.

#Solution for question a:

Mean and median for price

```
mean_price <- mean(housing_data$price, na.rm = TRUE)
median_price <- median(housing_data$price, na.rm = TRUE)
print(paste("Mean for price: ", mean_price, "Median for price: ",
median_price))
```

```
## [1] "Mean for price: 126698 Median for price: 123750"
```

Mean and median for Taxes

```
mean_taxes <- mean(housing_data$Taxes, na.rm = TRUE)
median_taxes <- median(housing_data$Taxes, na.rm = TRUE)
print(paste("Mean for taxes: ", mean_taxes, "Median for taxes: ",
median_taxes))
```

```
## [1] "Mean for taxes: 1668.2 Median for taxes: 1535"
```

#Solution for question b:

```
sd_price <- sd(housing_data$price, na.rm = TRUE)
sd_taxes <- sd(housing_data$Taxes, na.rm = TRUE)
print(paste("Standard Deviation of price: ", sd_price, "Standard Deviation of
price: ", sd_taxes))
```

```
## [1] "Standard Deviation of price: 56356.7747175586 Standard Deviation of  
price: 1009.28055184039"
```

#Solution for question c:

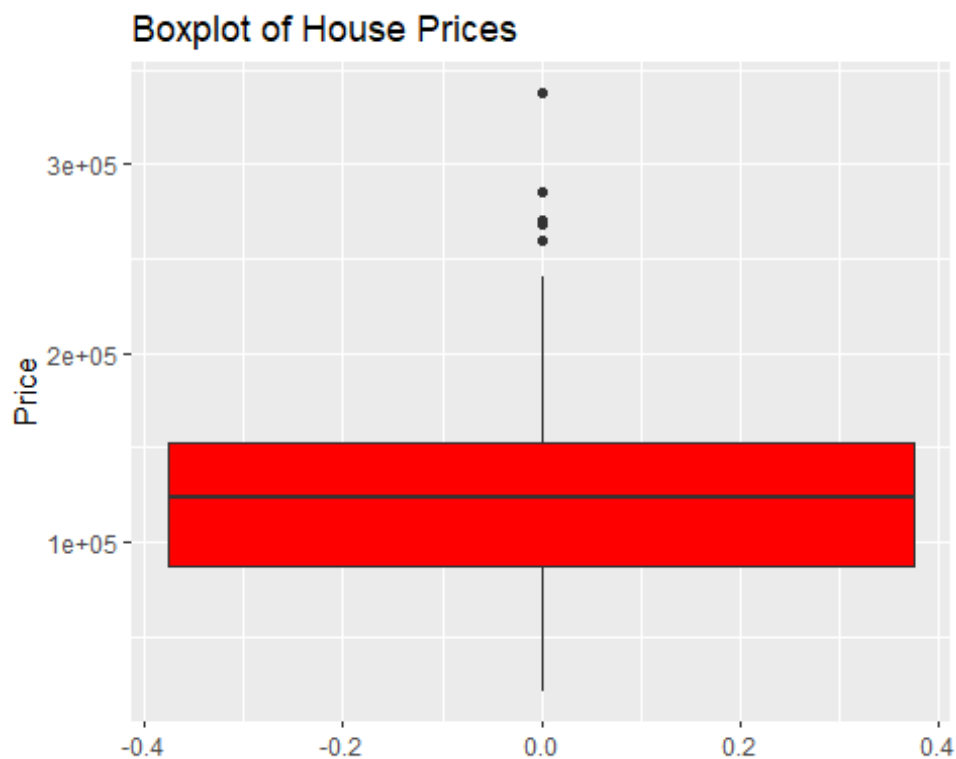
```
summary_price <- summary(housing_data$price)  
summary_taxes <- summary(housing_data$Taxes)  
print(paste("Summary of housing price: ",summary_price, "Summary of housing  
taxes: ", summary_taxes))
```

```
## [1] "Summary of housing price: 21000 Summary of housing taxes: 20"  
## [2] "Summary of housing price: 86875 Summary of housing taxes: 970"  
## [3] "Summary of housing price: 123750 Summary of housing taxes: 1535"  
## [4] "Summary of housing price: 126698 Summary of housing taxes: 1668.2"  
## [5] "Summary of housing price: 153075 Summary of housing taxes: 2042.5"  
## [6] "Summary of housing price: 338000 Summary of housing taxes: 4900"
```

#Solution for question d:

Boxplot for prices

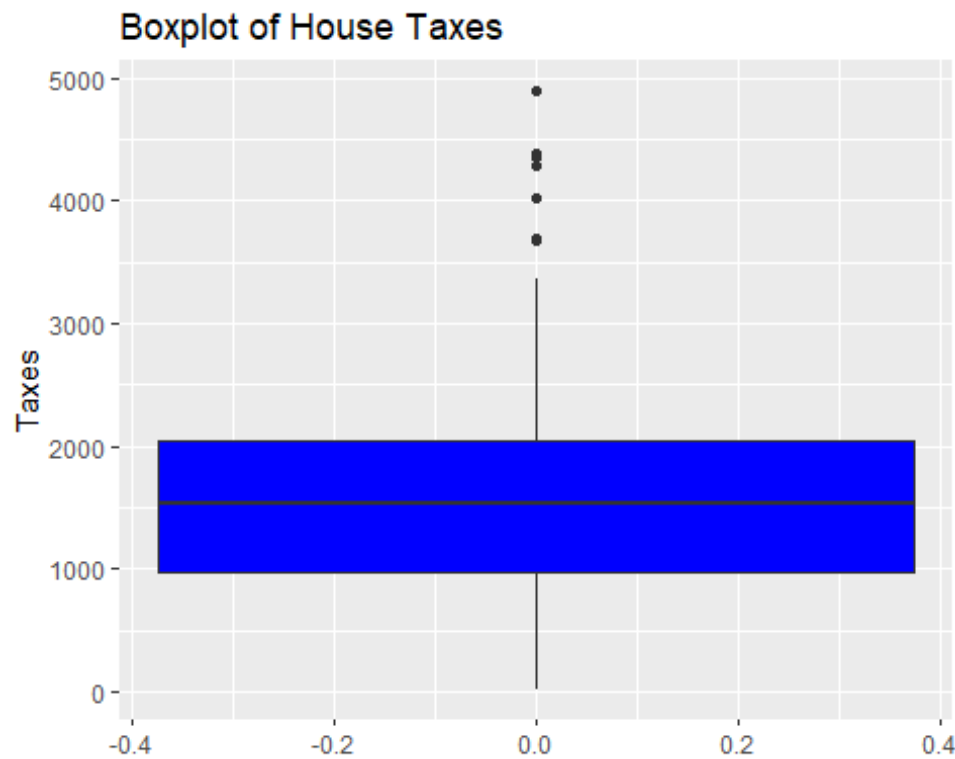
```
ggplot(housing_data, aes(y = price)) +  
  geom_boxplot(fill = "red") +  
  ggtitle("Boxplot of House Prices") +  
  ylab("Price")
```



Boxplot for taxes

```
ggplot(housing_data, aes(y = Taxes)) +  
  geom_boxplot(fill = "blue") +
```

```
ggtitle("Boxplot of House Taxes") +  
ylab("Taxes")
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.